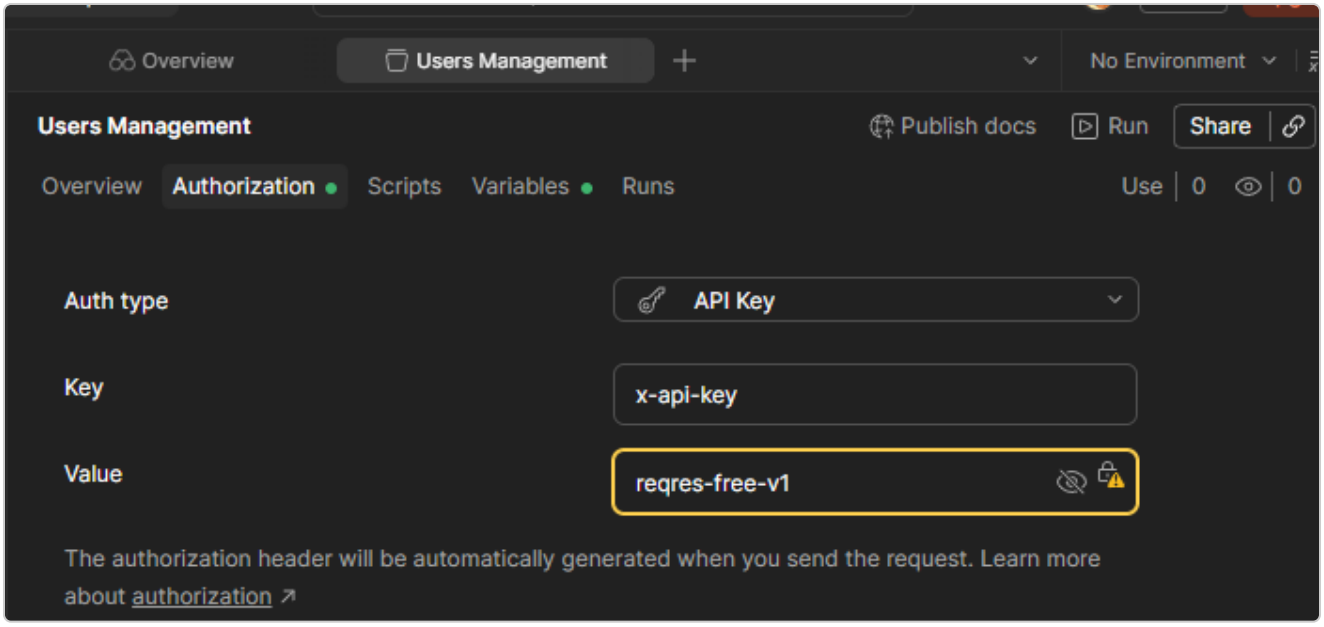
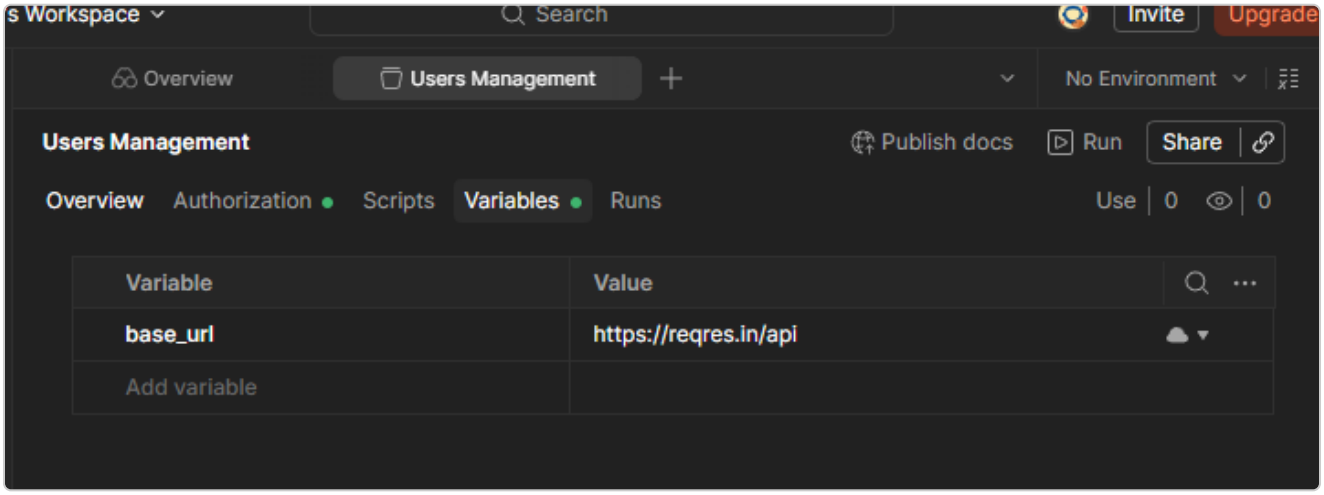
Practica T2: Gestio d'usuaris amb ReqRes

Proves d'una API REST amb Postman - https://reqres.in

Alumne	Esteve Amargant
Modul	OP1-B1-NODE
Colleccio Postman	Amargant_Esteve_T2_postman.json

1. Configuracio inicial de la col-leccio



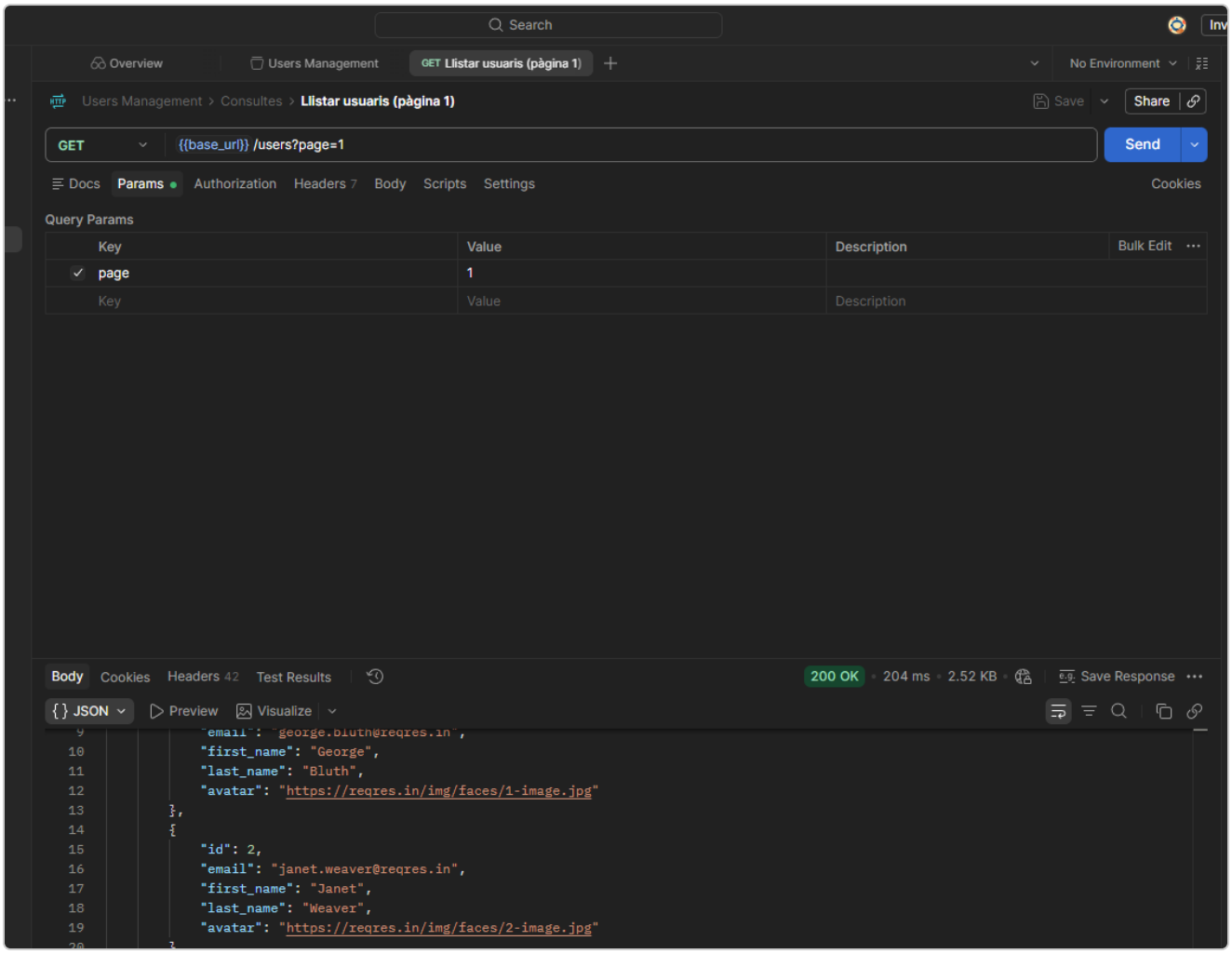


> Configuracio de la col-leccio "Users Management": a la pestanya **Variables** es defineix `base_url` amb el valor `https://reqres.in/api`, i a la pestanya **Authorization** es configura el tipus **API Key** amb Key `x-api-key` i Value `reqres-free-v1`, afegida a la capcalera (Add to: Header). Totes les peticions de la col-leccio hereten aquesta autoritzacio (Inherit auth from parent), de manera que la clau nomes es defineix una vegada.

2. Operacions CRUD

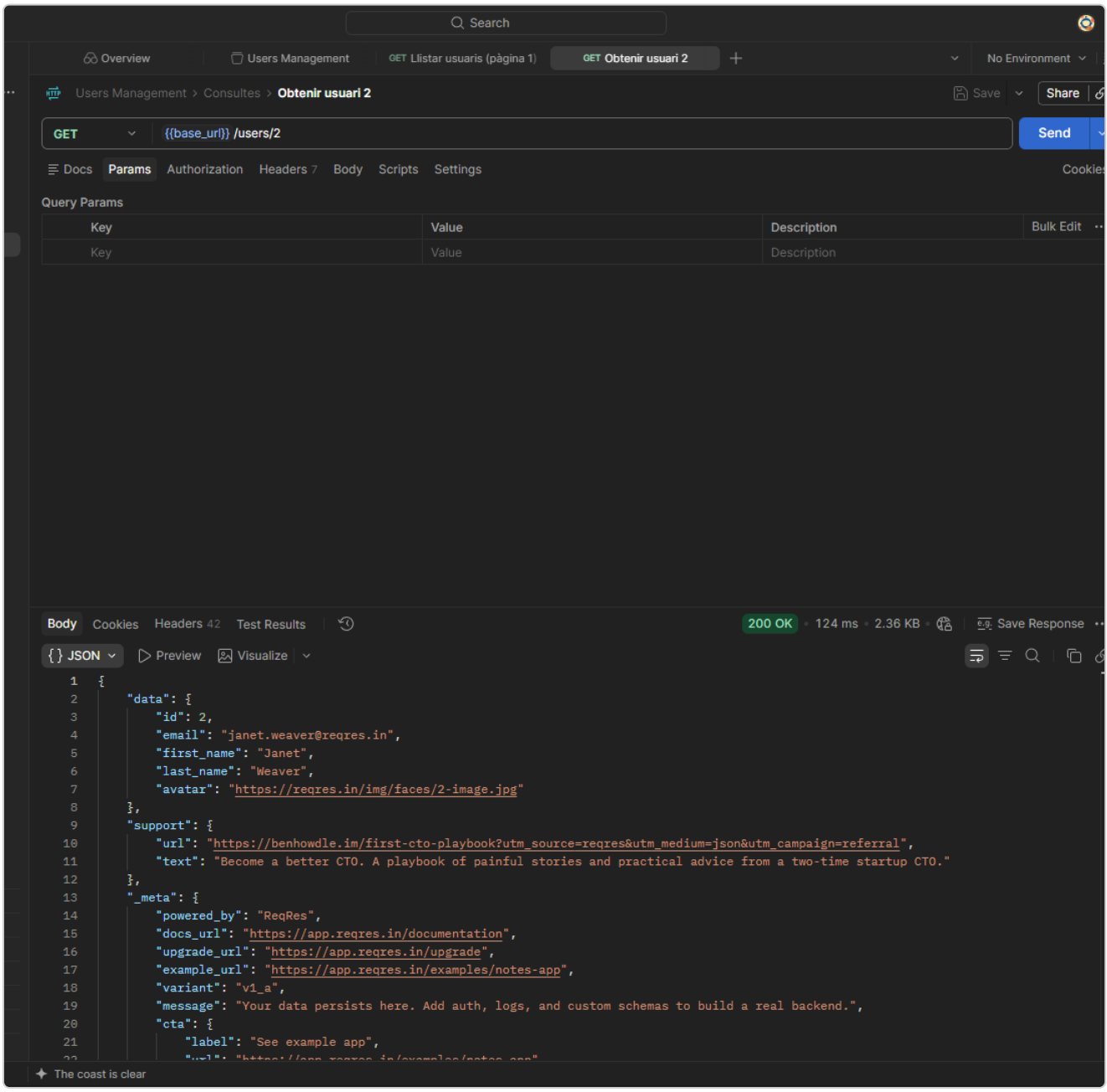
2.1 Consultes (GET)

GET base_url/users?page=1



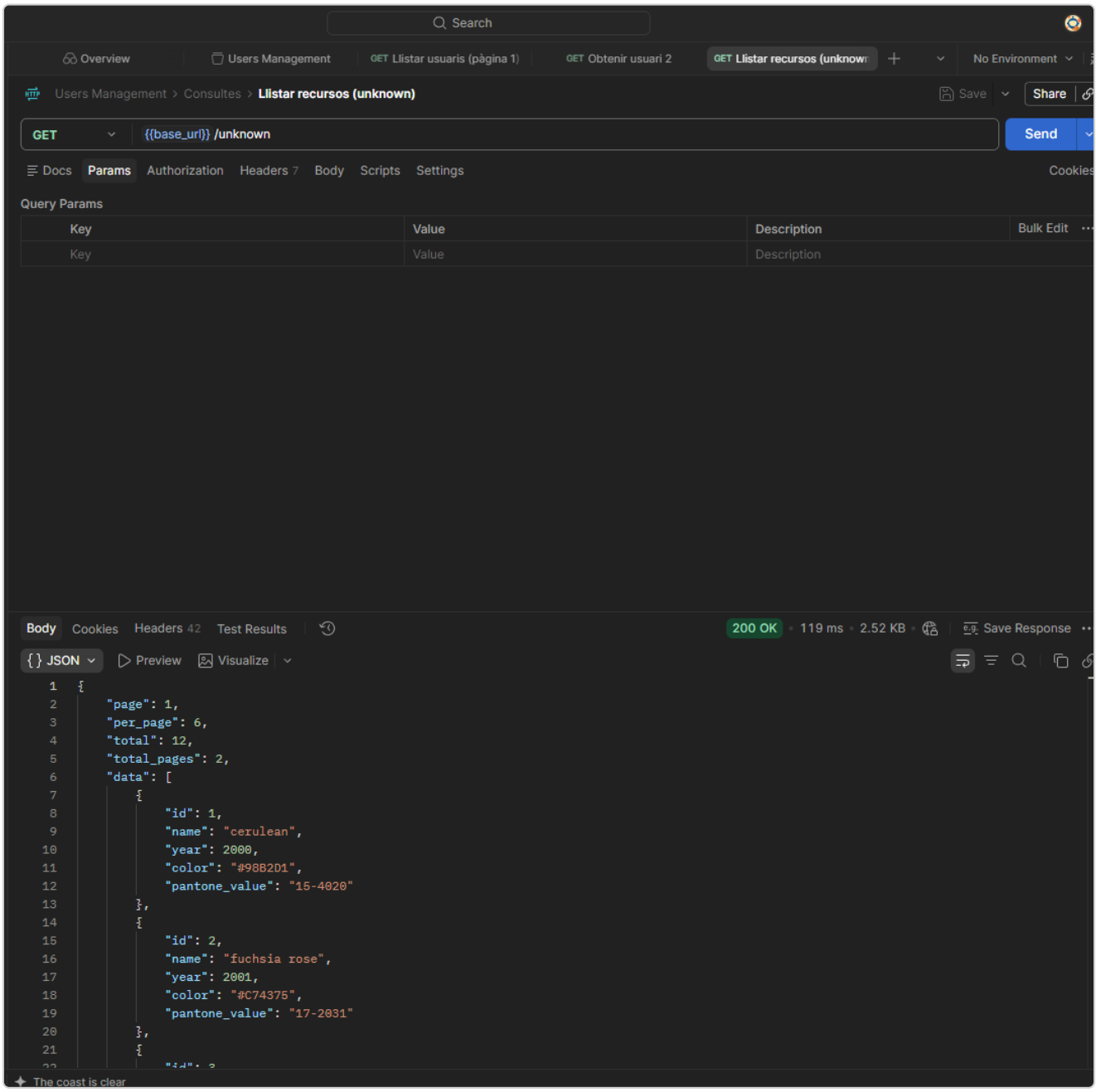
> Resposta **200 OK** en 204 ms. Retorna un objecte amb page, per_page, total, total_pages i l'array data amb els 6 usuaris de la primera pagina, cadascun amb id, email, nom, cognom i avatar.

GET base_url/users/2



> Resposta **200 OK** en 124 ms. Retorna un objecte data amb la informacio de l'usuari amb id 2 (Janet Weaver). La resposta tambe inclou un bloc addicional support/_meta amb informacio i enllacos promocionals del propi servei ReqRes, que no formen part de les dades de l'usuari.

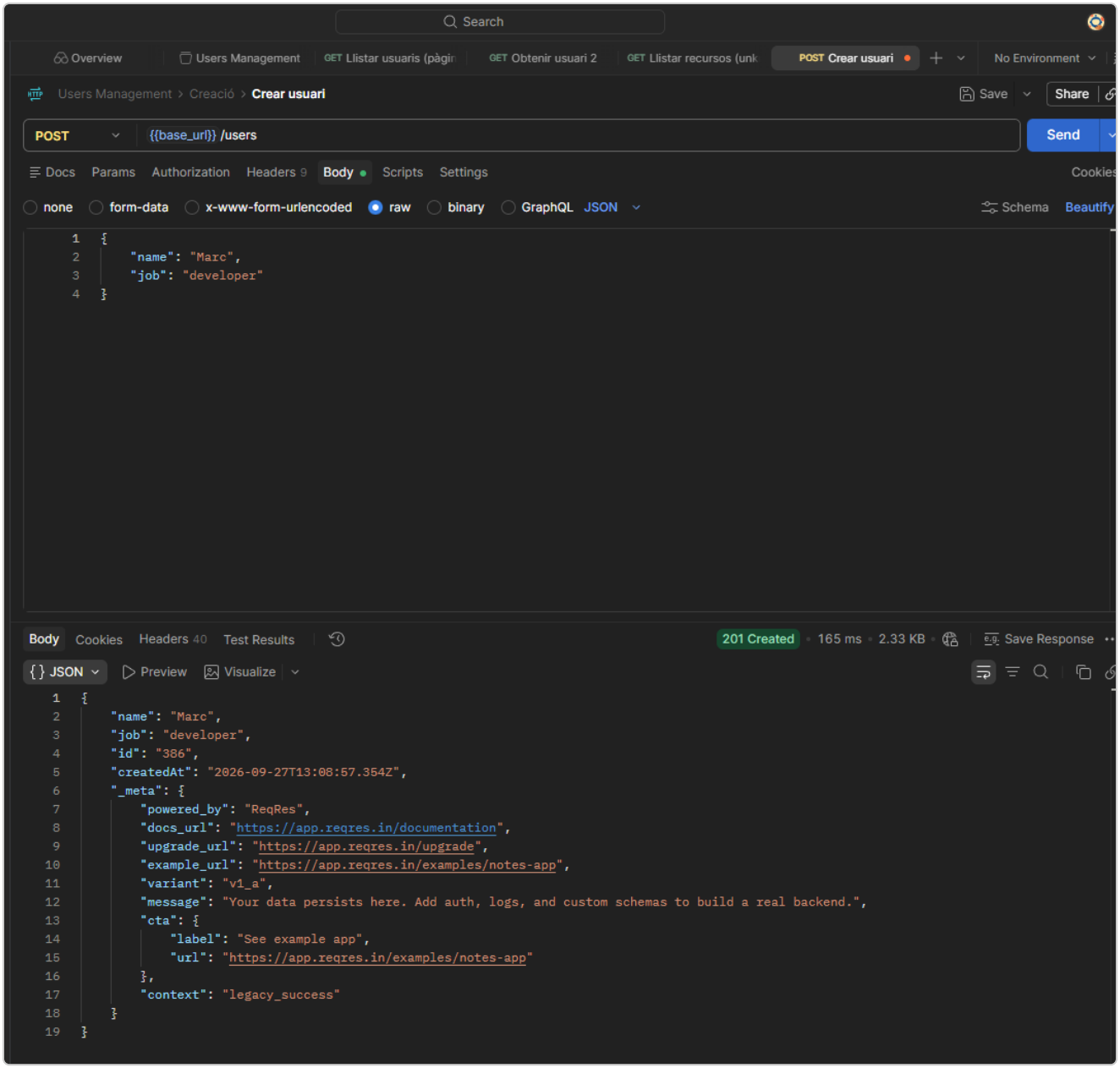
GET base_url/unknown



> Resposta **200 OK** en 119 ms. Mateixa estructura de paginacio que /users pero amb un altre recurs (colors): 6 elements per pagina amb id, name, year, color i pantone_value.

2.2 Creacio (POST)

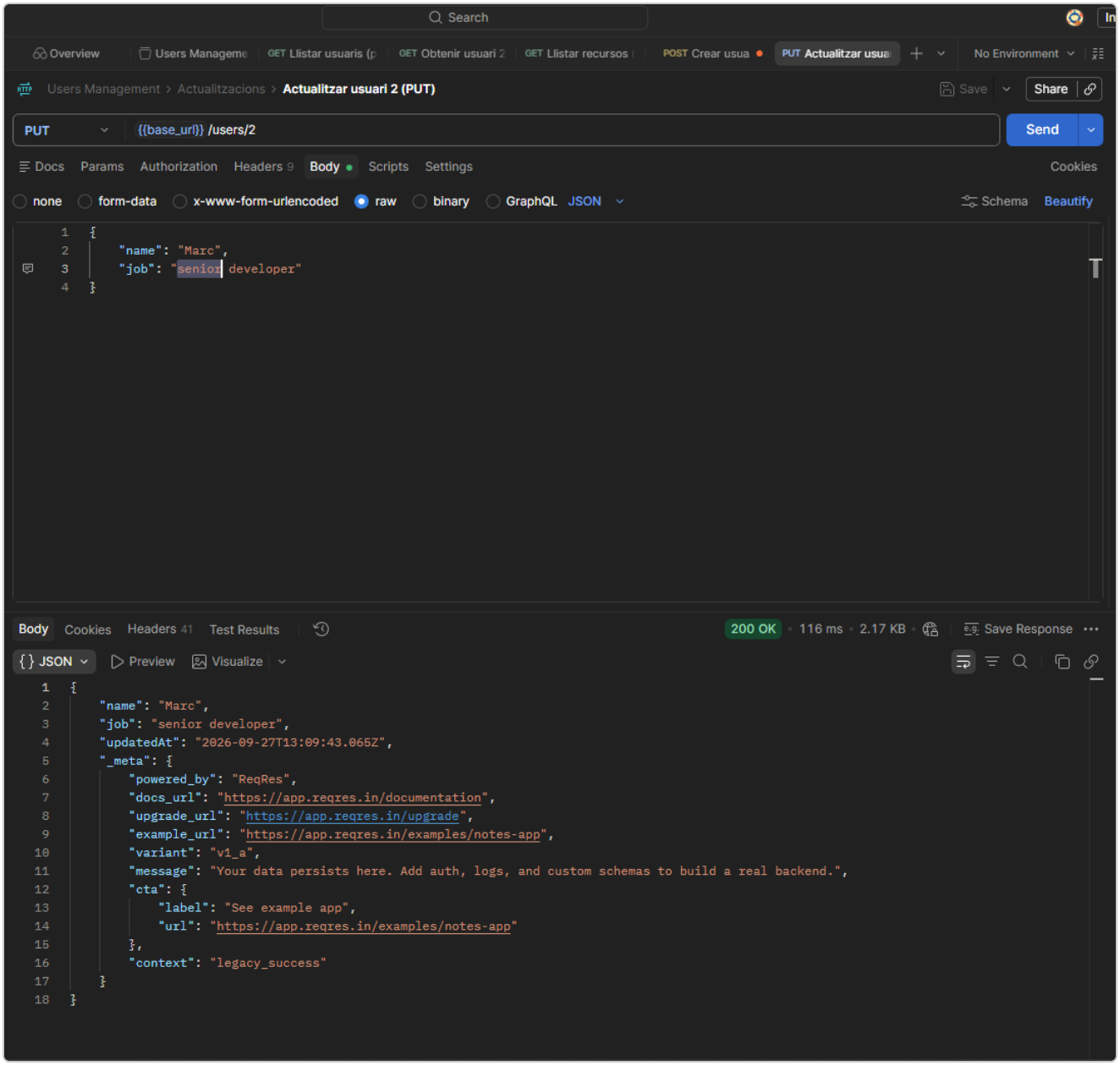
POST base_url/users



> Resposta **201 Created** en 165 ms. El body enviat (name, job) es retorna tal qual, i l'API hi afegeix un id generat i un createdAt amb la data i hora actuals. El camp context: "legacy_success" confirma que es tracta d'una simulacio de creacio compatible amb l'API classica de ReqRes.

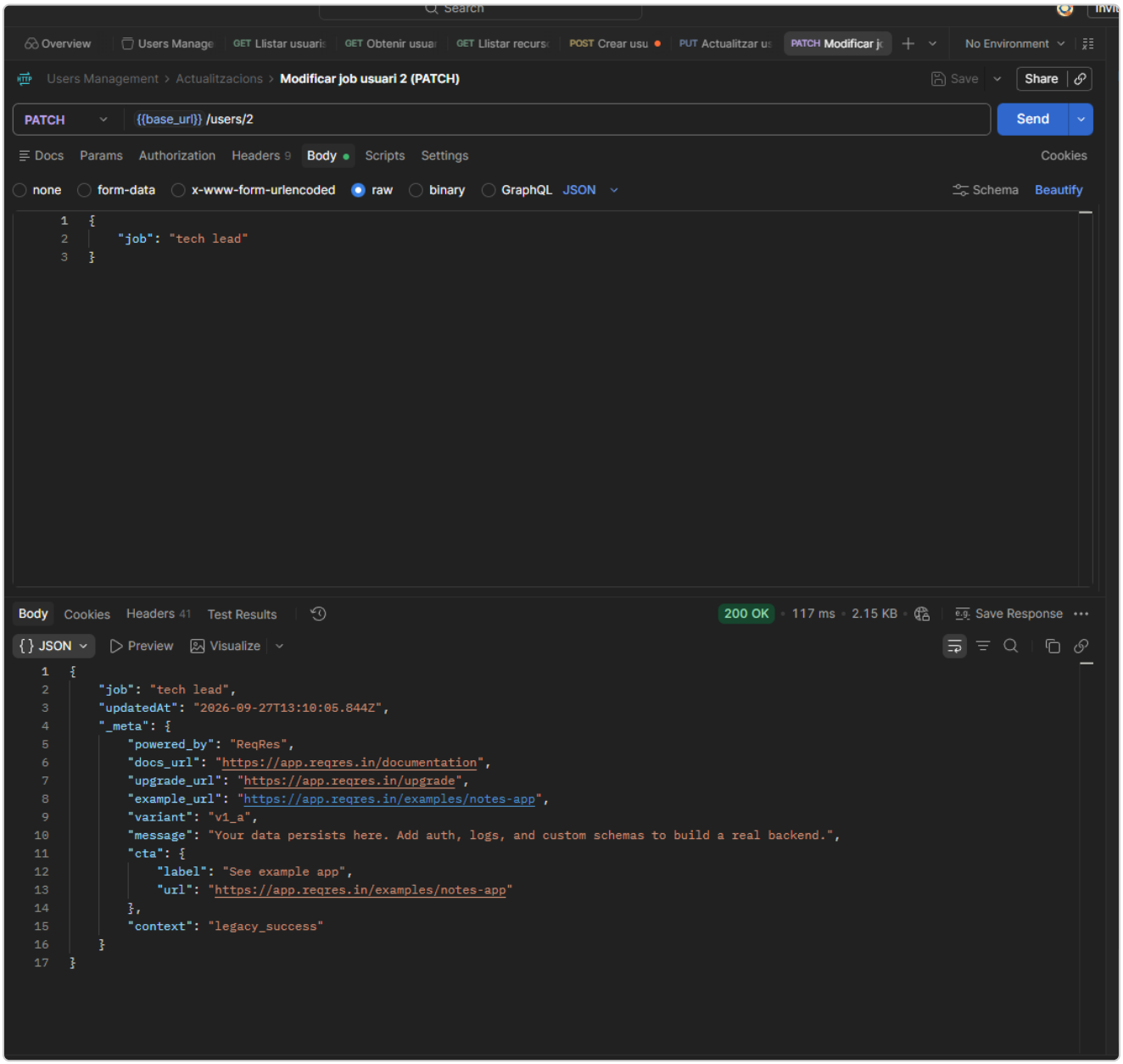
2.3 Actualitzacions (PUT i PATCH)

PUT base_url/users/2



> Resposta **200 OK** en 116 ms. Retorna el body enviat sencer (name, job actualitzat a "senior developer") mes un camp updatedAt amb el moment de la peticio.

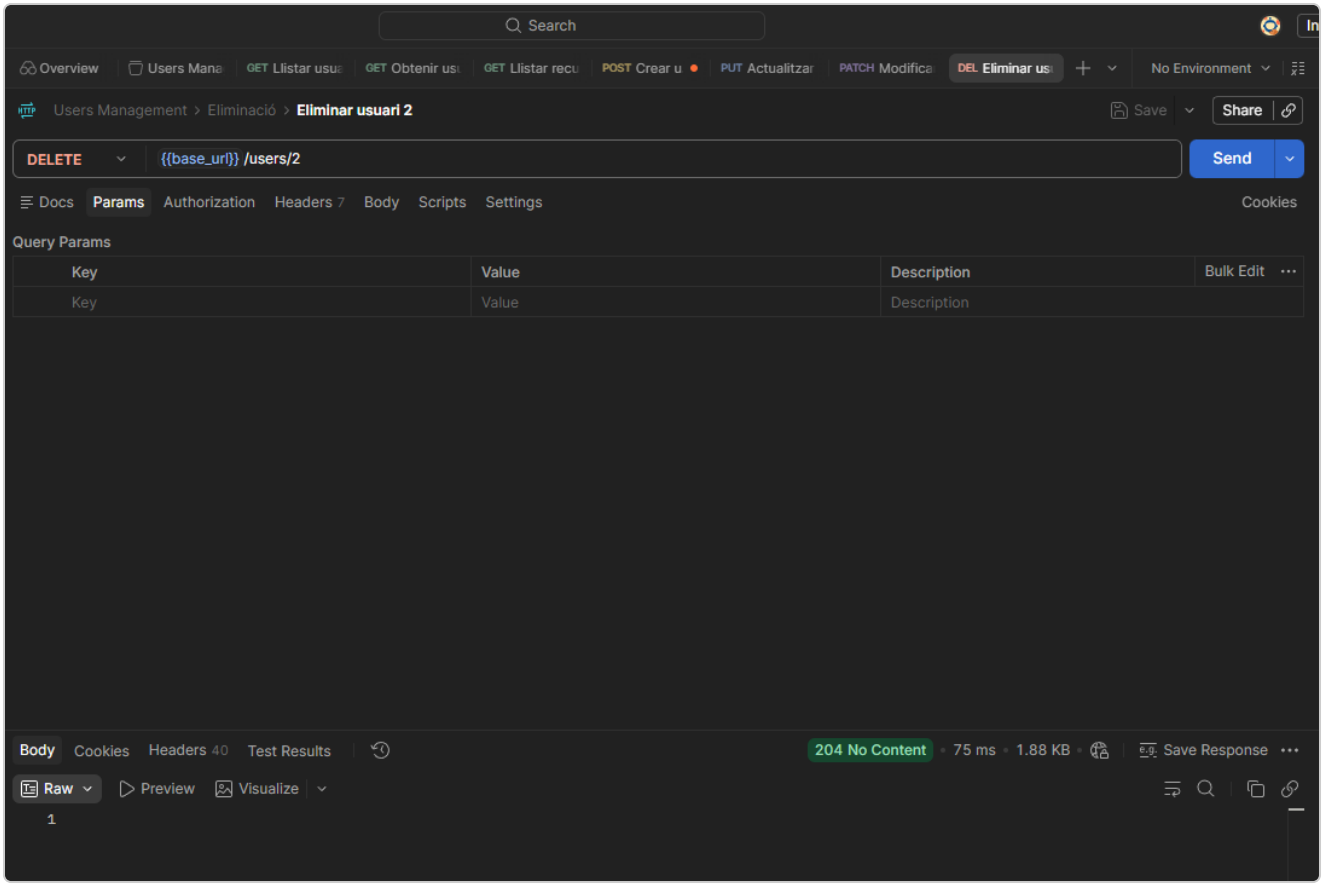
PATCH base_url/users/2



> Resposta **200 OK** en 117 ms. Nomes es va enviar el camp job ("tech lead") i la resposta el retorna junt amb un nou updatedAt, sense incloure el name perque no formava part del body enviat.

2.4 Eliminacio (DELETE)

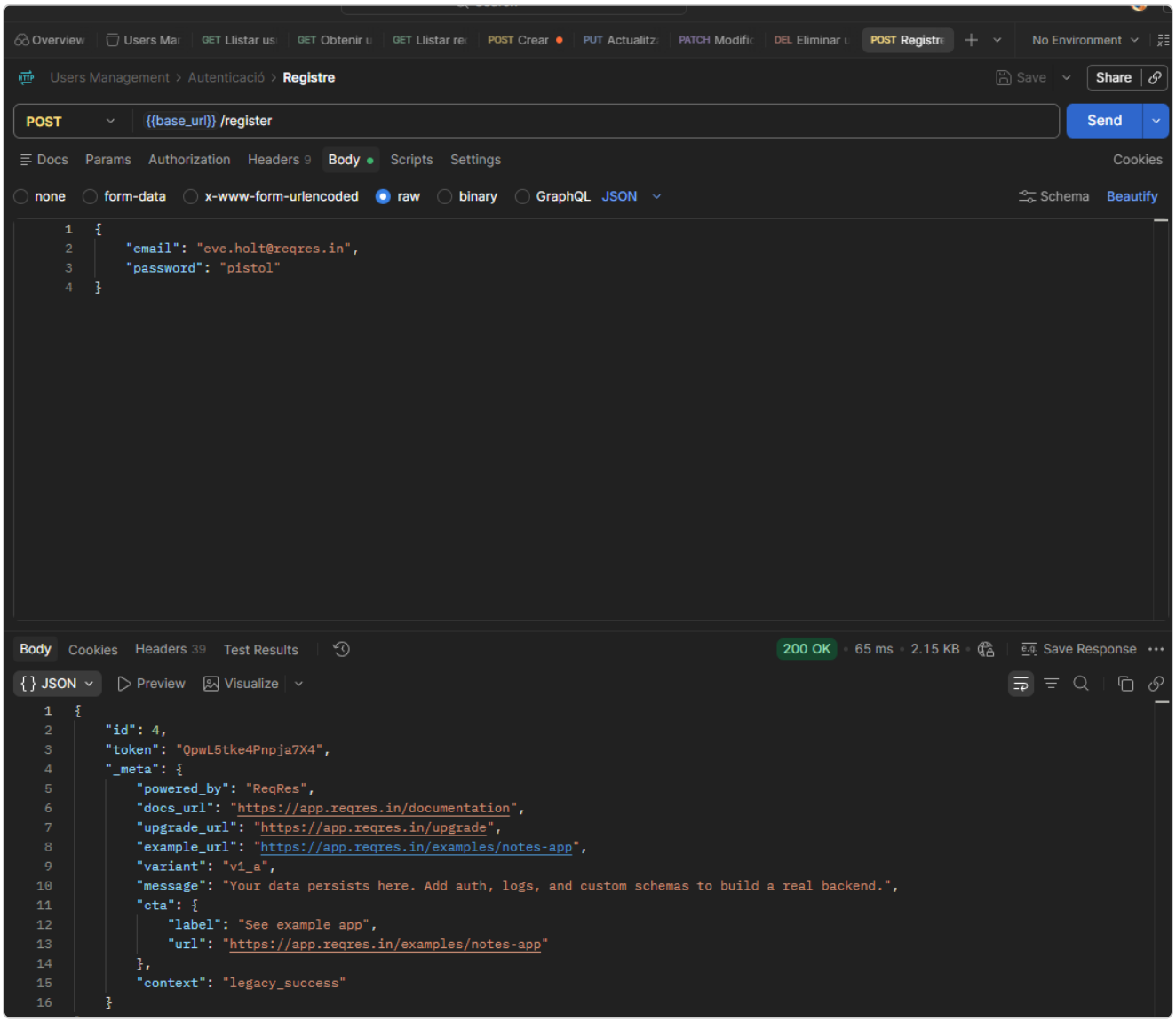
DELETE base_url/users/2



> Resposta **204 No Content** en 75 ms. El body de la resposta es buit: el codi 204 indica explicitament que l'operacio s'ha completat correctament pero que no hi ha contingut per retornar.

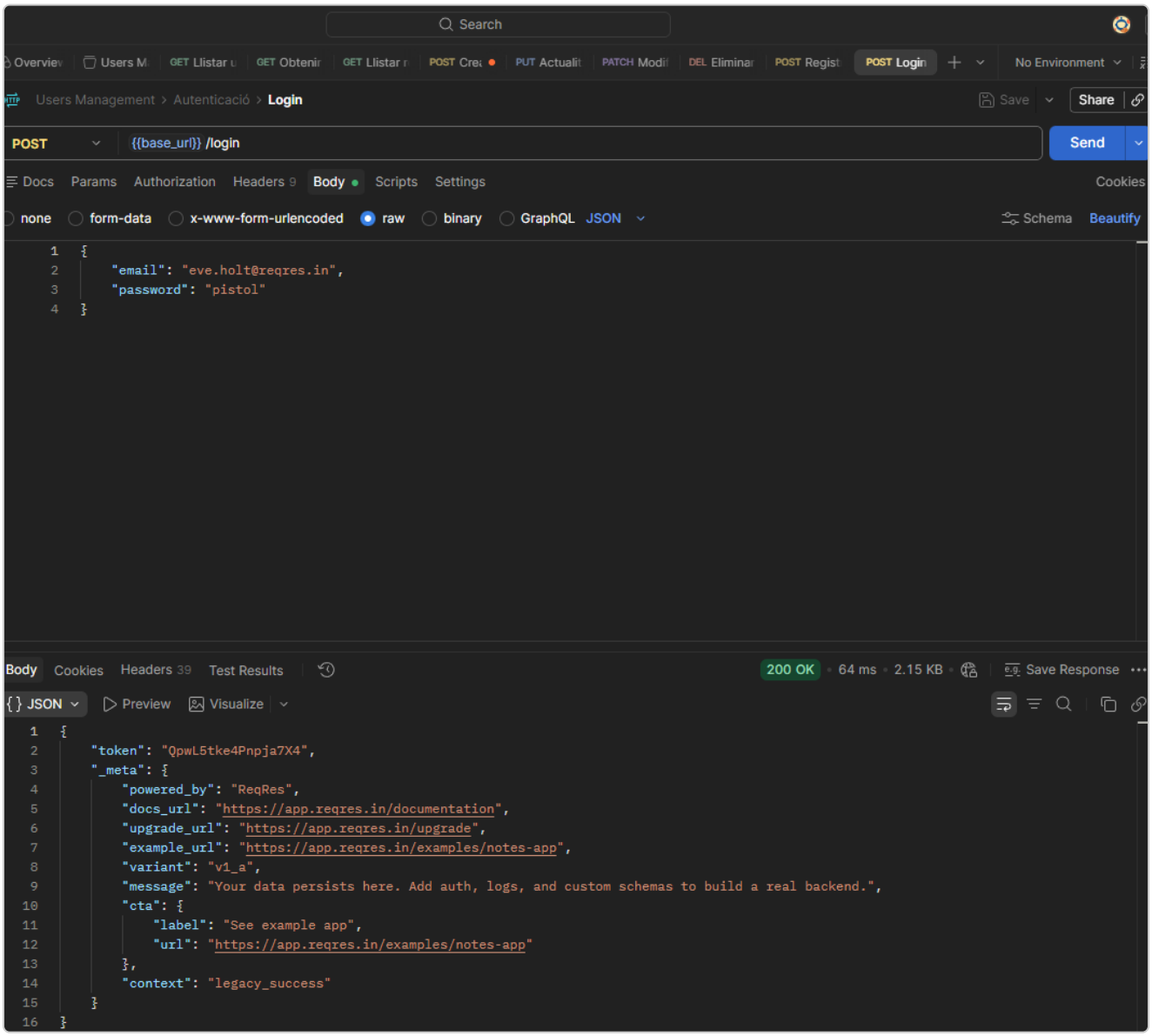
3. Autenticacio

POST base_url/register



> Resposta **200 OK** en 65 ms amb l'usuari de prova eve.holt@reqres.in / pistol. Retorna un id i un token de sessio simulat.

POST base_url/login

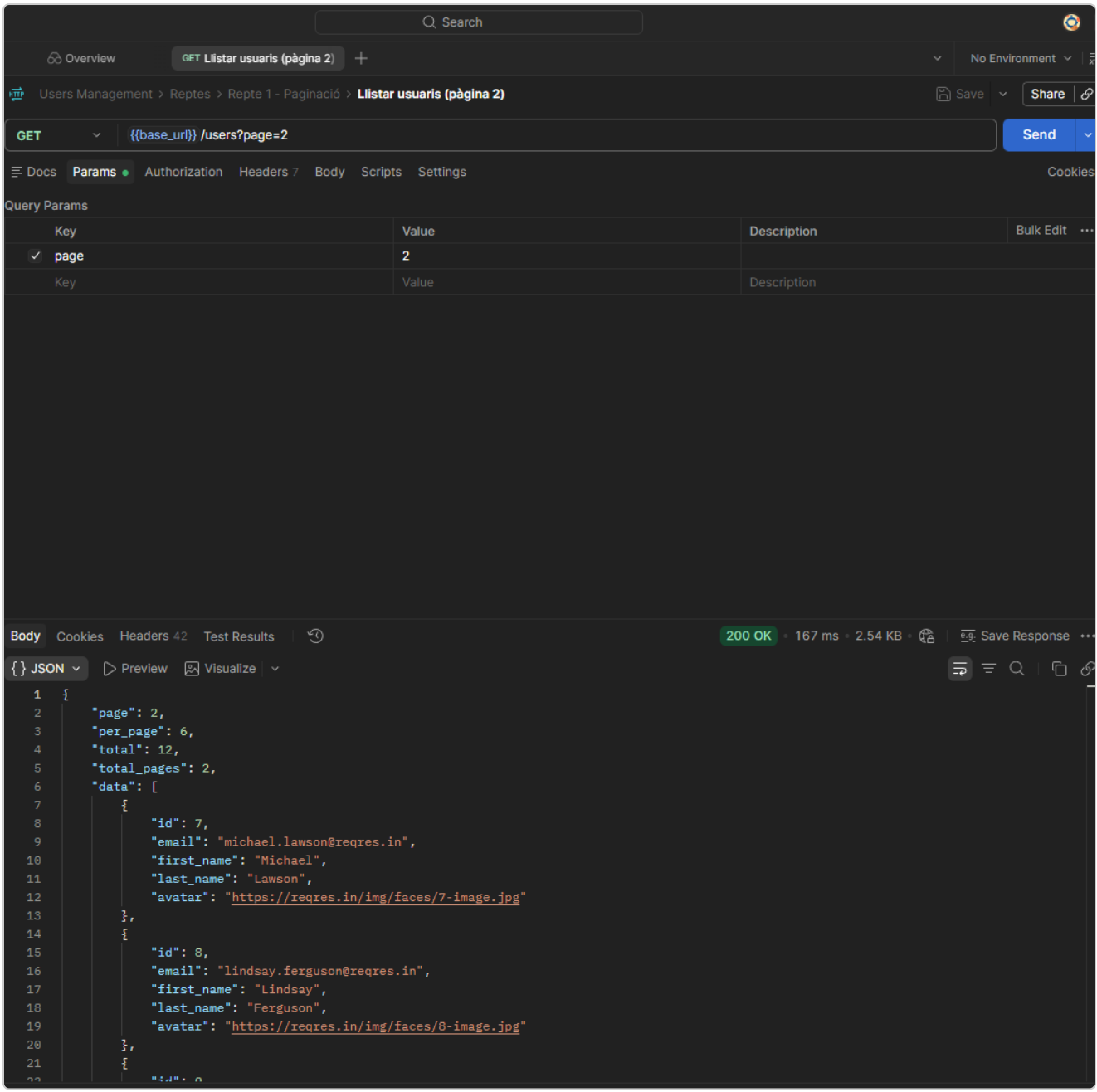


> Resposta **200 OK** en 64 ms amb les mateixes credencials. Retorna un token (el mateix valor que el del registre), que simula el token de sessio d'un usuari autenticat.

4. Reptes

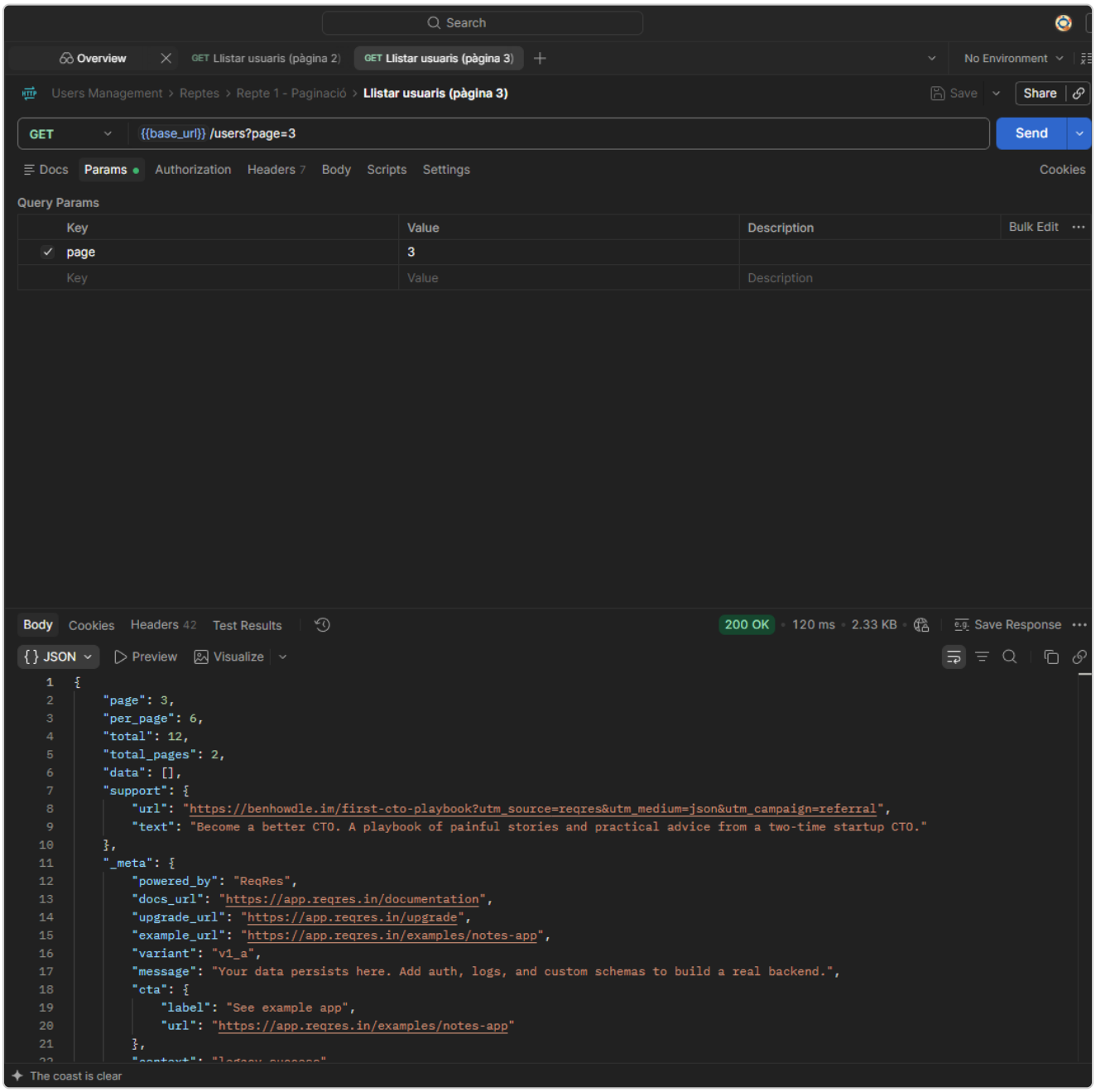
Repte 1: Paginacio

GET base_url/users?page=2



> Resposta **200 OK** en 167 ms amb els usuaris 7 a 12 (segona pagina de 6 elements). Els camps page (pagina actual), per_page (elements per pagina), total (total d'usuaris, 12) i total_pages (total de pagines, 2) descriuen la paginacio.

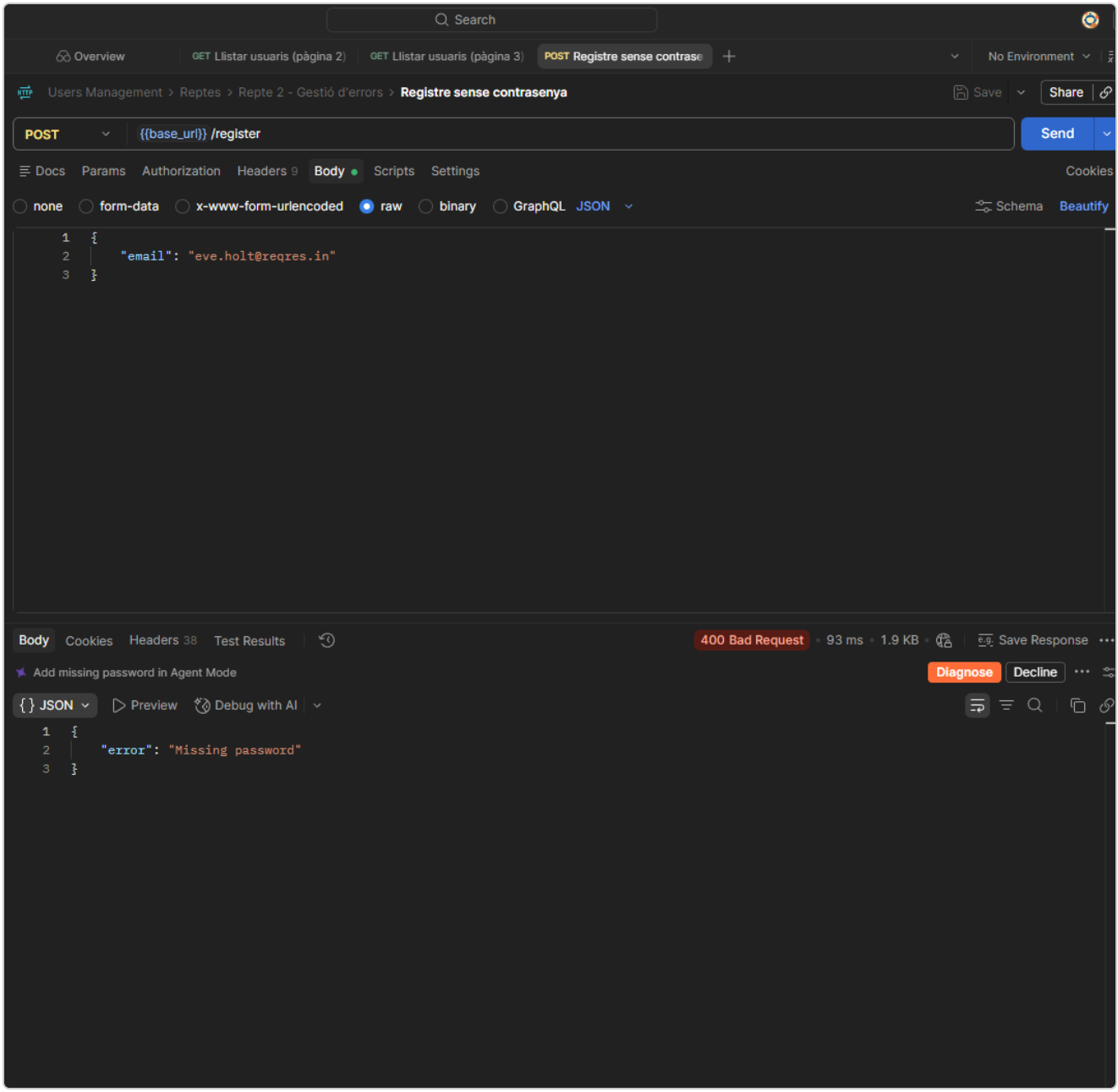
GET base_url/users?page=3



> Resposta **200 OK** en 120 ms, pero amb data: [] buit. No es un error perque la pagina 3 esta fora de rang (nomes hi ha 2 pagines), i l'API simplement retorna una llista buida en lloc d'un 404.

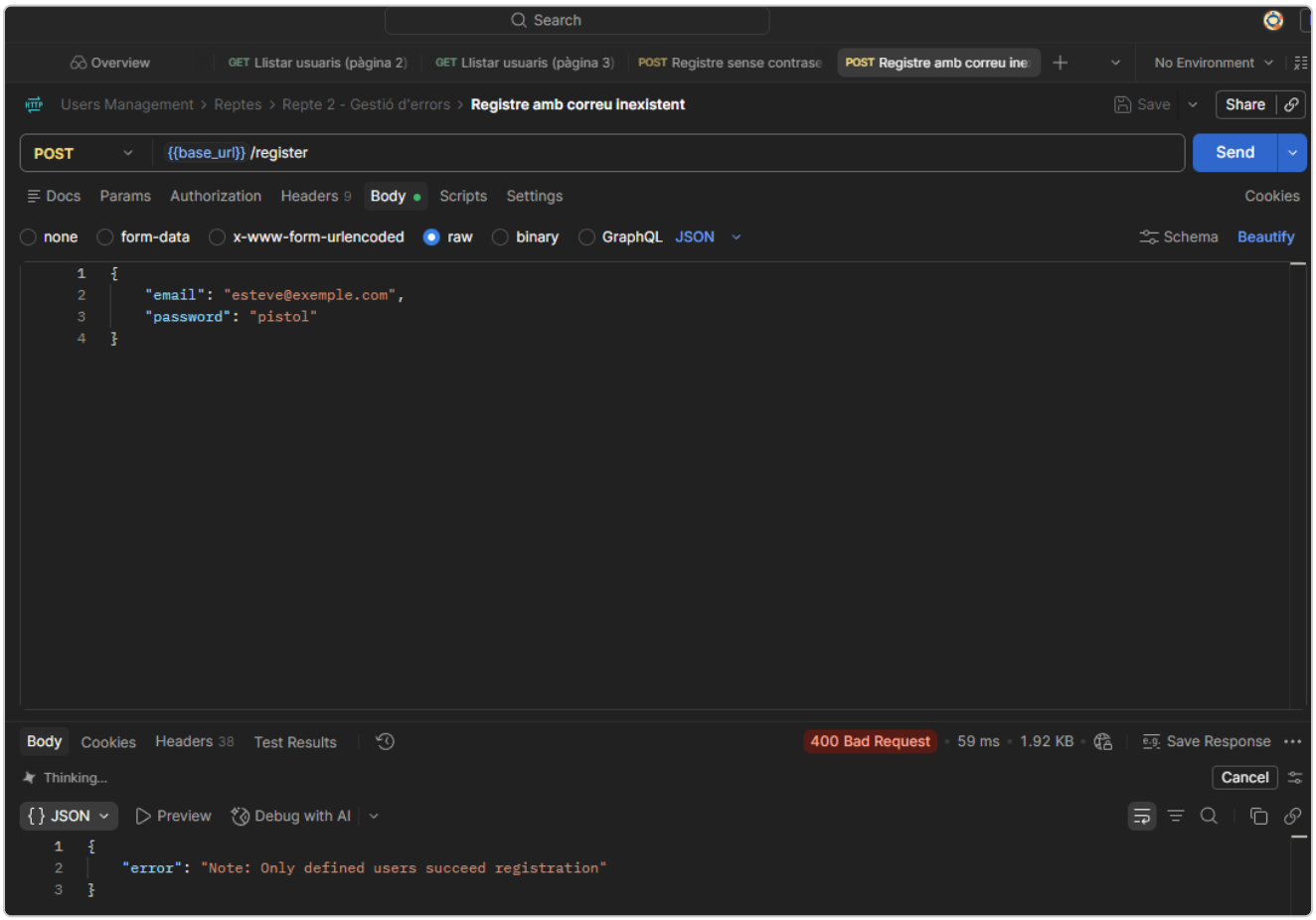
Repte 2: Gestio d'errors

POST base_url/register (sense contrasenya)



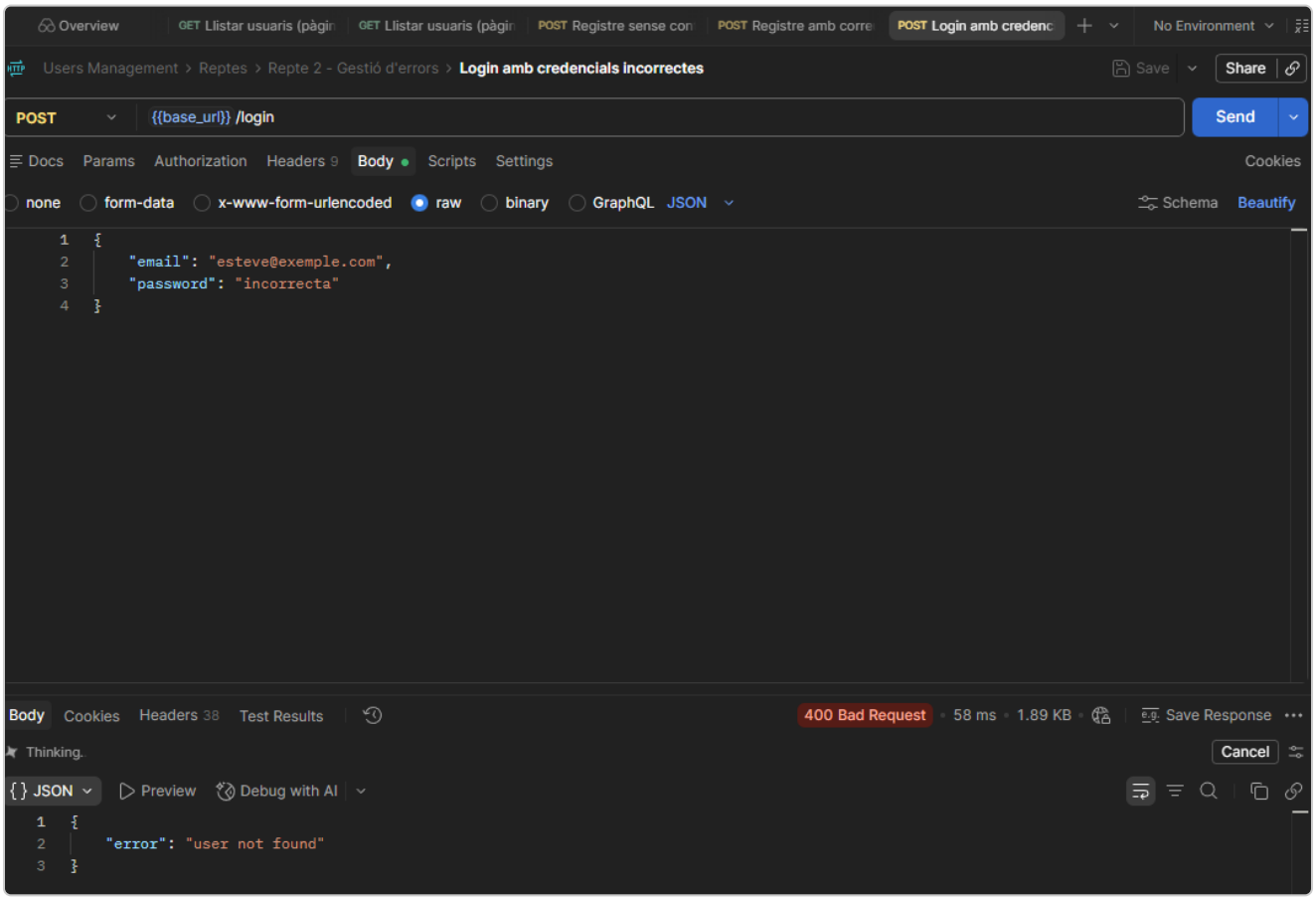
> **400 Bad Request** en 93 ms amb {"error": "Missing password"}. Falta un camp obligatori al body.

POST base_url/register (correu inexistent)



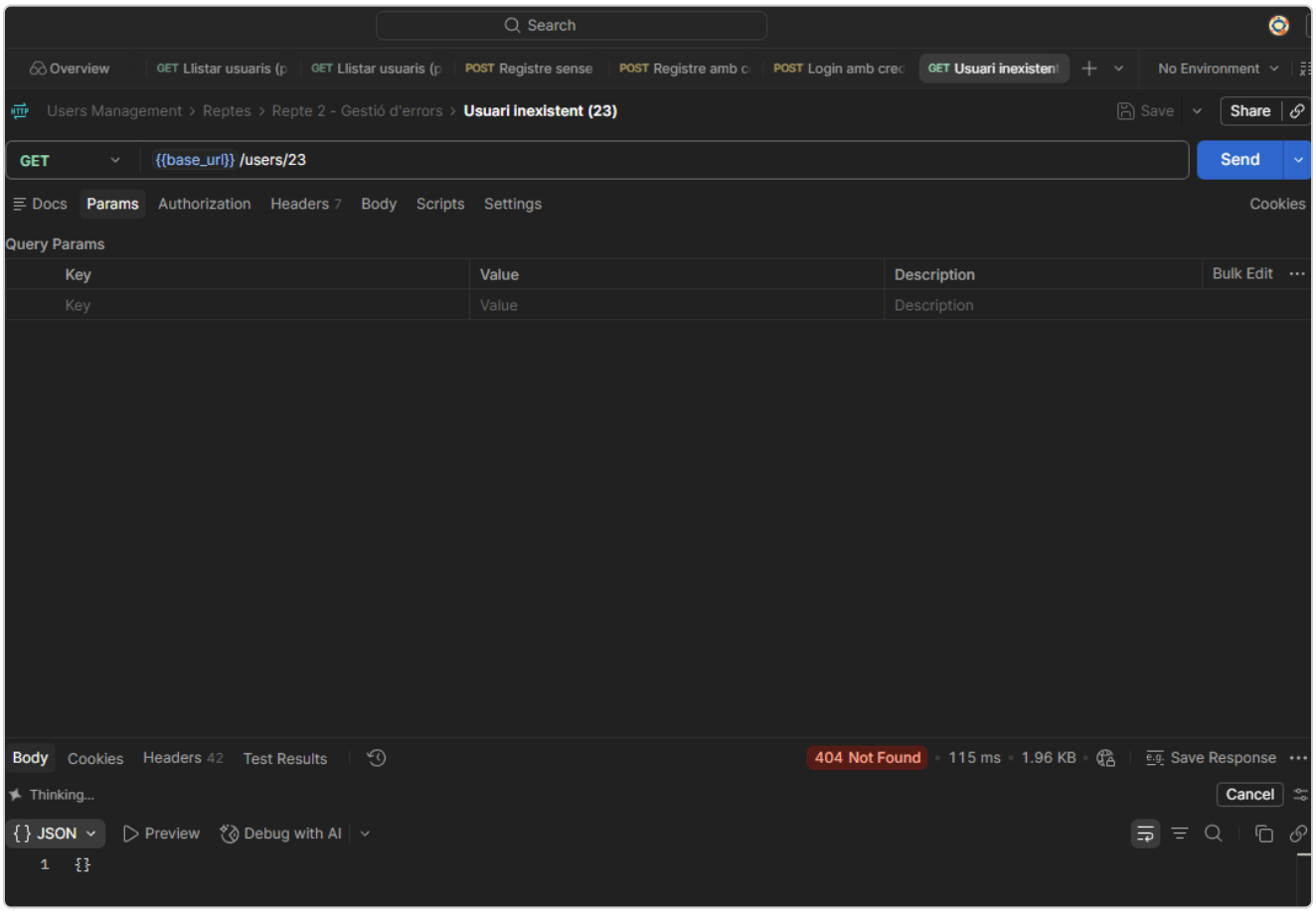
400 Bad Request en 59 ms amb {"error": "Note: Only defined users succeed registration"}. ReqRes nomes permet "registrar" els usuaris que ja existeixen a les seves dades de mostra (com eve.holt@reqres.in).

POST base_url/login (credencials incorrectes)



400 Bad Request en 58 ms amb {"error": "user not found"}. Ni l'email ni la contrasenya corresponen a cap usuari de mostra de l'API.

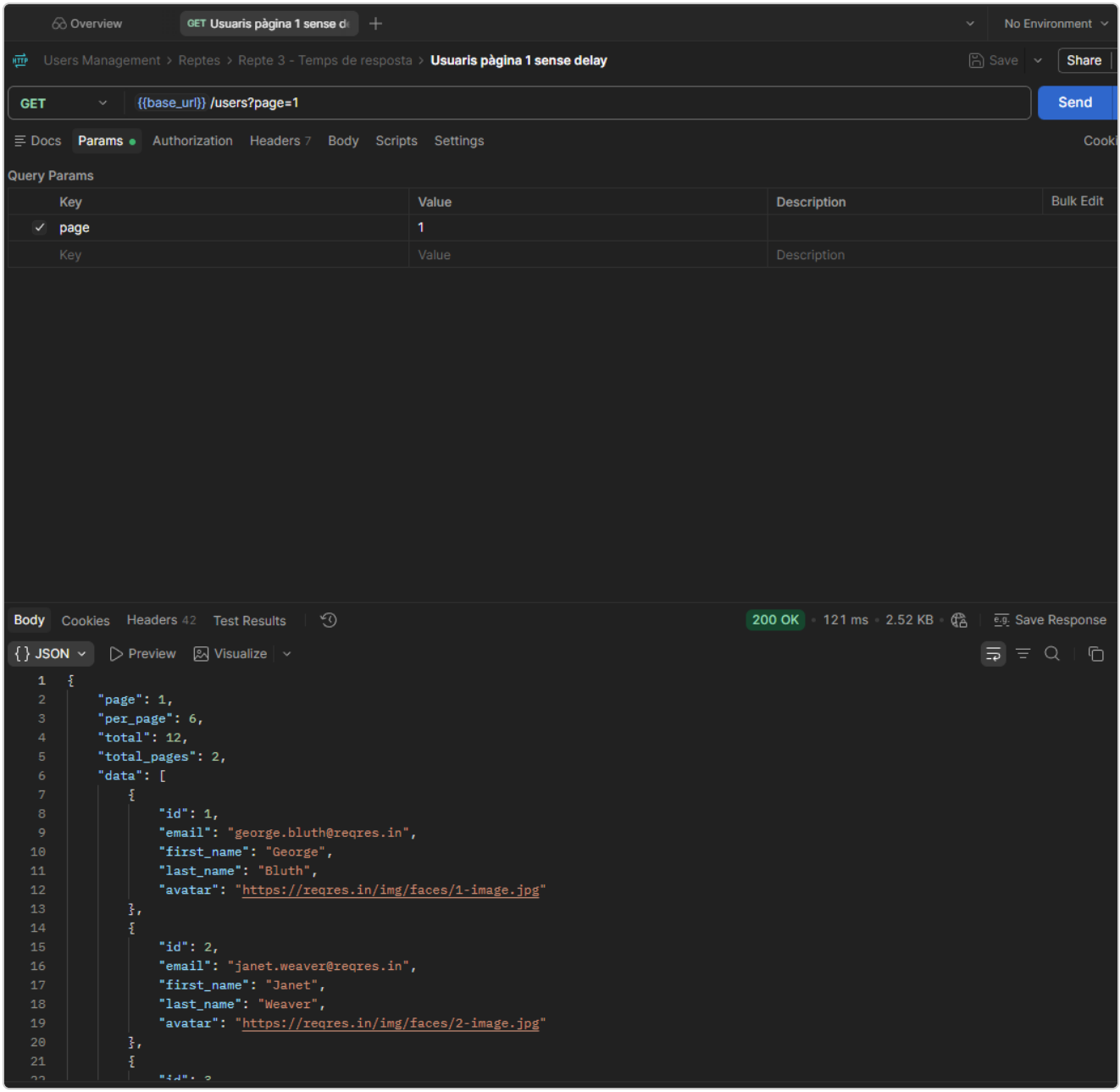
GET base_url/users/23



> **404 Not Found** en 115 ms amb un body buit {}. L'usuari amb id 23 no existeix a les dades de mostra de ReqRes.

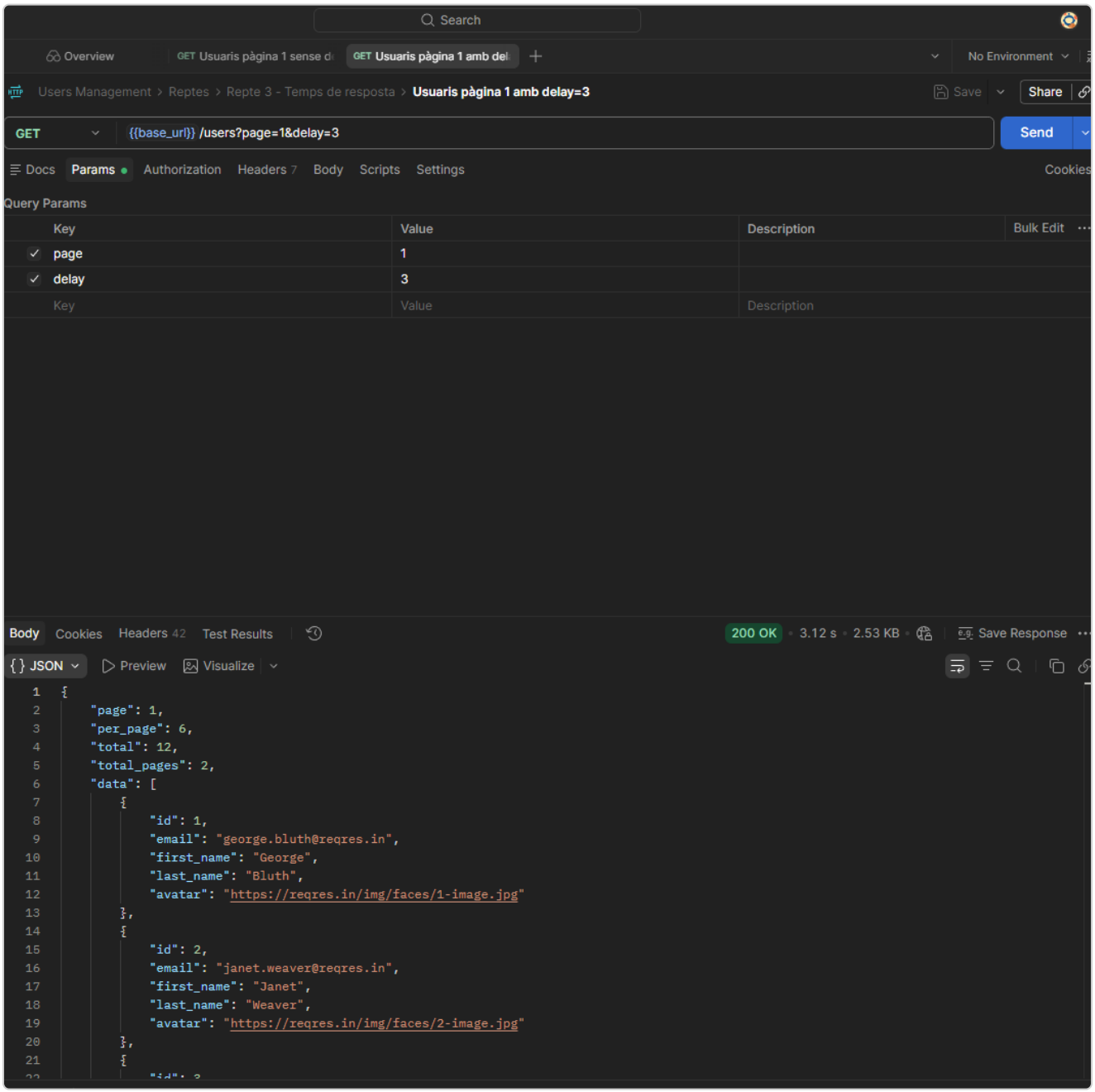
Repte 3: Temps de resposta

GET base_url/users?page=1 (sense delay)



> Resposta **200 OK** en **121 ms**, temps normal de resposta de l'API.

GET base_url/users?page=1&delay=3



> Resposta **200 OK** en **3,12 s** (uns 3 segons mes que sense el parametre). El parametre delay permet simular una API lenta, molt util per provar com reacciona una aplicacio (spinners, timeouts, estats de carrega) davant de respostes lentes sense dependre d'un servidor real carregat.

5. Preguntes d'analisi

1. Com gestiona ReqRes la paginacio? Quins camps de la resposta la descriuen i com sabries quantes peticions cal fer per obtenir tots els usuaris?

La resposta de /users inclou quatre camps de paginacio: page (la pagina demanada), per_page (elements que es retornen per pagina, 6), total (nombre total d'usuaris disponibles, 12) i total_pages (nombre total de pagines, 2). Per obtenir tots els usuaris nomes cal dividir

total entre per_page (o llegir directament total_pages) i anar incrementant el parametre page d'1 fins a total_pages; en aquest cas caldrien 2 peticions (page=1 i page=2).

2. Les respostes de PUT i PATCH a ReqRes son gairebe identiques. Quina diferencia hi hauria entre els dos metodes en una API real que deses les dades?

A ReqRes tots dos retornen el body enviat mes un updatedAt, perque cap dels dos persisteix res. En una API real, PUT hauria de reemplacar el recurs sencer: si s'oblida algun camp al body, aquell camp s'esborraria o es posaria a un valor per defecte. PATCH, en canvi, nomes modificaria els camps inclosos al body, deixant la resta del recurs intacte. Aquesta diferencia no es veu a ReqRes perque no hi ha una base de dades real darrere que conservi l'estat entre peticions.

3. Despres de crear un usuari amb POST, el pots obtenir amb GET? Per que? Que ens diu aixo de ReqRes com a API de proves?

No, un cop creat l'usuari amb POST (per exemple amb id "386" i createdAt), si es fa un GET a /users/386 l'API respon 404, ja que no existeix a les dades de mostra. Aixo confirma el que indica l'enunciat: ReqRes no desa res realment, nomes simula la resposta esperada d'una API real (retornant les dades enviades mes els camps que generaria un backend real, com id o createdAt) sense persistir-les enlloc. Aixo la fa ideal per practicar el format de les peticions i respostes, pero no per provar fluxos que depenguin de dades persistents entre peticions.

4. Per que la resposta del DELETE no te contingut? Quin codi d'estat ho indica?

El codi **204 No Content** indica explicitament que la peticio s'ha processat correctament pero que no hi ha res per retornar al client. Te sentit en un DELETE: un cop l'element s'ha eliminat, no queda cap recurs sobre el qual informar, aixi que el servidor respon amb un cos buit en lloc de tornar a enviar les dades de l'usuari eliminat.

5. Quins codis d'estat i quins missatges retorna ReqRes quan el registre o el login fallen? Et semblen prou clars per mostrar-los a l'usuari d'una aplicacio?

En tots els casos provats el codi es **400 Bad Request**, amb un camp error que descriu la causa: "Missing password" (falta la contrasenya al registre), "Note: Only defined users succeed registration" (correu no reconegut al registre) i "user not found" (credencials incorrectes al login). Son missatges clars per a un desenvolupador que esta depurant la crida, pero no son directament adequats per mostrar-los tal qual a un usuari final d'una aplicacio: caldria traduir-los i adaptar-los (per exemple, "Introdueix una contrasenya" o "Email o contrasenya incorrectes") per no exposar detalls interns de l'API ni donar pistes a un atacant sobre si un email existeix o no al sistema.